

BytePing: Computing from the Heart

A Theoretical Framework for Efficient Data Marshalling and Unmarshalling in Duplex Streams

Obinexus Computing

July 12, 2026

Abstract

In the era of distributed systems and real-time data processing, efficient serialization mechanisms are paramount. This paper introduces **BytePing**, a theoretical framework for marshalling and unmarshalling data in duplex streams using buffering techniques. BytePing aims to optimize bandwidth utilization, ensure data integrity, and facilitate seamless integration with high-level languages.

1 Introduction

The proliferation of data-intensive applications necessitates efficient data serialization methods. Traditional serialization frameworks like Protocol Buffers (Protobuf) offer structured data representation but may not be optimized for streaming scenarios. BytePing addresses this gap by providing a chunk-based marshalling system tailored for duplex streams.

2 System Architecture

BytePing’s architecture comprises the following components:

- **Buffer Manager:** Segments the data stream into manageable chunks.
- **Hasher:** Computes hash values for each chunk to ensure data integrity.
- **Marshaller:** Combines data chunks with their corresponding hash values.
- **Duplex Stream Handler:** Manages bidirectional data flow, facilitating simultaneous read and write operations.

3 Data Flow Mechanism

The data flow in BytePing follows these steps:

1. **Data Ingestion:** Raw data is received and segmented into fixed-size buffers.
2. **Hash Computation:** Each buffer undergoes a hashing process to generate a unique identifier.
3. **Chunk Formation:** The buffer and its hash are combined to form a data chunk.
4. **Streaming:** Chunks are transmitted over the duplex stream, allowing for concurrent sending and receiving.
5. **Unmarshalling:** On the receiving end, chunks are verified using their hash values and reassembled into the original data stream.

4 Advantages of BytePing

- **Bandwidth Efficiency:** By transmitting only necessary data chunks and avoiding redundant information, BytePing reduces bandwidth consumption.
- **Data Integrity:** Hash-based verification ensures that data corruption during transmission is detected and mitigated.
- **Real-Time Processing:** Duplex streaming allows for immediate data processing, essential for time-sensitive applications.
- **Language Agnosticism:** While the core is implemented in C for performance, BytePing provides bindings for high-level languages like Python, facilitating broader adoption.

5 Potential Applications

BytePing's design makes it suitable for various applications:

- **IoT Devices:** Efficient data transmission between constrained devices and central servers.
- **Live Streaming:** Real-time audio and video data transmission with minimal latency.
- **Distributed Databases:** Synchronization of data across distributed nodes with integrity checks.
- **Edge Computing:** Processing data at the edge with efficient serialization mechanisms.

6 Conclusion

BytePing offers a novel approach to data serialization in duplex streams, emphasizing efficiency, integrity, and adaptability. Future work includes developing a comprehensive implementation, benchmarking against existing frameworks like Protobuf, and exploring integration with various programming environments.

